Squeezing Every FLOP: Single-GPU Training Efficiency Optimization for Vision-Language Models

A Case Study on SiQ-VL with NVIDIA Blackwell

Duo An
duo.an@example.com

May 2026

Abstract

We present a systematic study of single-GPU training efficiency optimization for SiQ-VL, a vision-language model trained on NVIDIA RTX PRO 6000 Blackwell (96 GB VRAM). Through 15 iterations of measured optimizations—spanning dtype fixes, kernel fusion (Liger-Kernel, TileGym cuTile), data strategies (bucketing, packing), and batch scaling—we improve throughput from **4,674 to 107,367** real tokens/second: a **22.9× end-to-end speedup**. We document both successes and failures: vision feature caching (0% gain), Triton Flash Attention (slower than SDPA), FlexAttention packing (51% waste), and TileGym with variable shapes (5× regression). The final configuration achieves MFU≈22.7% on the 0.5B model, confirmed to be near the memory-bandwidth ceiling for this model scale.

Keywords: Vision-Language Models, Training Efficiency, GPU Kernel Optimization, cuTile, Liger-Kernel, Blackwell, Sequence Packing

Contents

1 Introduction

Training vision-language models (VLMs) efficiently on a single GPU is crucial for researchers with limited hardware budgets. While much attention has been paid to distributed training at scale, the *single-GPU efficiency frontier* determines the baseline throughput that distributed methods multiply.

This report documents the complete optimization journey of **SiQ-VL**, a compact VLM combining a SigLIP-2 vision encoder with a Qwen2.5 language model. We focus exclusively on engineering optimizations that preserve model quality (identical loss curves) while maximizing tokens processed per second.

1.1 Optimization Timeline

The optimization proceeded through 15 iterations over two model configurations:

1. **Phase A** (Iter 0–10): SigLIP2-so400m + Qwen2.5-1.5B (589M+494M params). Baseline 4,674 tok/s → 28,322 tok/s (**6.06**×).
2. **Phase B** (Iter 11–15): SigLIP2-base-224 + Qwen2.5-0.5B (92.9M+494M params). Focus on kernel optimization (TileGym cuTile). Baseline 21,673 tok/s → 107,367 tok/s (**4.95**×).

Combined end-to-end: **4,674 → 107,367 = 22.9**× (across model scales).

1.2 Contributions

1. A **reproducible benchmark** (`benchmark_real_efficiency.py`) that measures all metrics on actual training data.
2. Comprehensive evaluation of **Liger-Kernel vs TileGym cuTile**: TileGym is 30–41% faster with 50–64% less VRAM on Blackwell.
3. Demonstration that **length bucketing + large batch** renders sequence packing marginal (+4.6%) for uniform-length datasets.
4. Documentation of **6 failed optimizations** with root-cause analysis.
5. Evidence that 0.5B models are **memory-bandwidth bound** at 19–23% MFU, independent of kernel or data strategy.

2 System Setup

Table 1: Hardware and model configurations

Hardware	
GPU	NVIDIA RTX PRO 6000 Blackwell Server Edition
VRAM / Bandwidth	96 GB GDDR7 / 1.79 TB/s
Compute	188 SMs @ 2430 MHz, Peak BF16: 936 TFLOP/s
CUDA / Driver	CUDA 13.0, Driver 580.126
Model (Phase B — final configuration)	
Vision Encoder	SigLIP2-base-patch16-224 (92.9M, frozen)
Language Model	Qwen2.5-0.5B-Instruct (494.0M)
Projector	2-layer MLP + Pixel Shuffle (2.8M)
Total	589.7M parameters
Software	
Framework	PyTorch 2.9.1, Transformers 5.3.0
Kernels	TileGym (cuTile DSL), Liger-Kernel 0.8.0
Dataset	sharegpt4v/coco (50K samples)

3 The Optimization Journey

3.1 Iteration 0: The FP32 Baseline (4,674 tok/s)

The initial profiler trace revealed a critical bug: despite declaring `bf16=True`, 49.96% of CUDA time was spent in `cutlass_80_simt_sgemm` (FP32 GEMM kernels). Root cause: `from_pretrained()` loads weights in FP32 by default; HF Trainer's autocast only covers the forward pass, not the model weights themselves.

3.2 Iteration 1: bf16 Fix + Vision no_grad (14,366 tok/s, 3.07×)

Loading with `torch_dtype=torch.bfloat16` and wrapping the frozen vision encoder in `torch.no_grad()` gave an immediate **3× speedup** and 40% VRAM reduction. *Lesson: always verify actual kernel dispatches, not just config flags.*

3.3 Iteration 2: Liger-Kernel (11,070 tok/s, ↓25% speed, ↓41% VRAM)

We integrated Liger-Kernel's fused operators (Linear-CE, RMSNorm, SwiGLU, RoPE). **Paradoxically, this was slower** in Stage 1 because the LLM is frozen—fused CE backward savings don't materialize when there's no backward through the LLM. However, VRAM dropped 41% (from 11.5 to 6.8 GB) because the fused CE never materializes the full $(B, L, 151936)$ logits tensor.

Lesson: kernel fusion benefits depend on which parts of the model have gradients. Stage 1 (projector-only) doesn't benefit from LLM kernel fusion speed-wise.

3.4 Iteration 3: Batch Size Scaling (20,284 tok/s, 4.34×)

With the VRAM headroom from Liger, we scaled from B=4 to B=16. This 4× batch increase yielded only 1.83× throughput gain because larger matmuls approach the memory-bandwidth ceiling. Investigation results:

Table 2: Approaches investigated in Iteration 3

Approach	Tok/s	VRAM	Verdict
Vision feature caching	11,070	8.6 GB	Worse (H2D > compute)
torch.compile on vision	11,070	6.8 GB	Marginal (+3%)
Gradient checkpointing off	11,400	6.8 GB	Marginal (+3%)
Batch size 4→16	**20,284**	**16.5 GB**	**Winner (+83%)**

3.5 Iteration 5: Length Bucketing (21,026 tok/s, 4.50×)

HuggingFace `group_by_length` reduced padding waste from 17.4% to 4.1%. Despite eliminating 13 percentage points of waste, tok/s improved only 3.7% because the dataset (sharegpt4v/coco) already has low length variance (CoV=11.5%).

3.6 Iteration 7: torch.compile Replaces Liger (27,352 tok/s, 5.85×)

A key discovery: **Liger-Kernel and torch.compile are mutually exclusive**. Liger's Triton patches cause illegal memory access under `torch._dynamo` tracing. Replacing Liger with `torch.compile(mode="max-autotune-no-cudagraphs")` gave **30% speedup** over Liger (but lost the VRAM savings).

Lesson: whole-graph compilation outperforms targeted operator fusion for this workload.

3.7 Iteration 8: Batch Tuning Under Compile (28,322 tok/s, 6.06×)

Pushed B from 16 to 20 (+3.5%). Further increases showed diminishing returns because `torch.compile` recompiles on shape changes, and variable-length sequences already produce multiple discrete shapes per epoch.

3.8 Iteration 10: Packing + FlexAttention (26,787 tok/s)

On a mixed 6-subset dataset (length variance σ=400), packing into N=1536 bins reduced padding from 10.2% to ∼2%, yielding +4.8% throughput and -18% VRAM. The gain was modest because bucketing already captured most variance.

3.9 Iteration 11: TileGym FA4 — Blackwell-Native Attention

NVIDIA's TileGym provides cuTile DSL kernels targeting Blackwell hardware features (wgmma, TMA, SM remapping). Key result for Flash Attention:

Table 3: TileGym FA4 vs SDPA (end-to-end Stage 1 step time)

Backend	N=512	N=1024	N=1536
SDPA	69.2 ms	82.1 ms	84.3 ms
TileGym FA4	64.9 ms	75.2 ms	75.0 ms
Speedup	**1.07×**	**1.09×**	**1.12×**

The speedup comes from FA4's **native GQA support**: Qwen2.5's 14Q/2KV config requires SDPA to expand K/V 7× via `repeat_kv`, while FA4 handles it in-kernel.

3.10 Iteration 13: TileGym Full Stack (30–41% faster than Liger)

The complete TileGym replacement (`apply_tilegym_kernel_to_qwen2`) patches ALL Qwen2 ops: RoPE, RMSNorm, SwiGLU, FA4 attention, plus our custom fused linear CE.

Table 4: TileGym full stack vs Liger-Kernel (Stage 2, gradient checkpointing, B=4)

N	Liger tok/s	TileGym tok/s	Speed Δ	VRAM Δ
512	38.3K	49.7K	**+30%**	**−50%**
1024	46.5K	64.6K	**+39%**	**−59%**
2048	48.7K	68.7K	**+41%**	**−64%**

Fused Linear Cross-Entropy with Grad-in-Forward. Our custom `torch.autograd.Function` computes ∇_h and ∇_W *within* the forward loop using TileGym's `_ce_cutile` kernel. The backward is $O(1)$ (scalar scaling). This reduces VRAM by 51–64% for the LM head in Stage 2.

3.11 Iteration 15: Ground Truth Verification (107K tok/s)

All previous numbers used profiler estimates or synthetic data. Iteration 15 re-measured everything on **real data** with precise `attention_mask` accounting. Peak verified throughput: **107,367 Real tok/s** (Packing + TileGym, B=64→23 packed bins, N=1024).

4 Results

4.1 The Full Speedup Progression

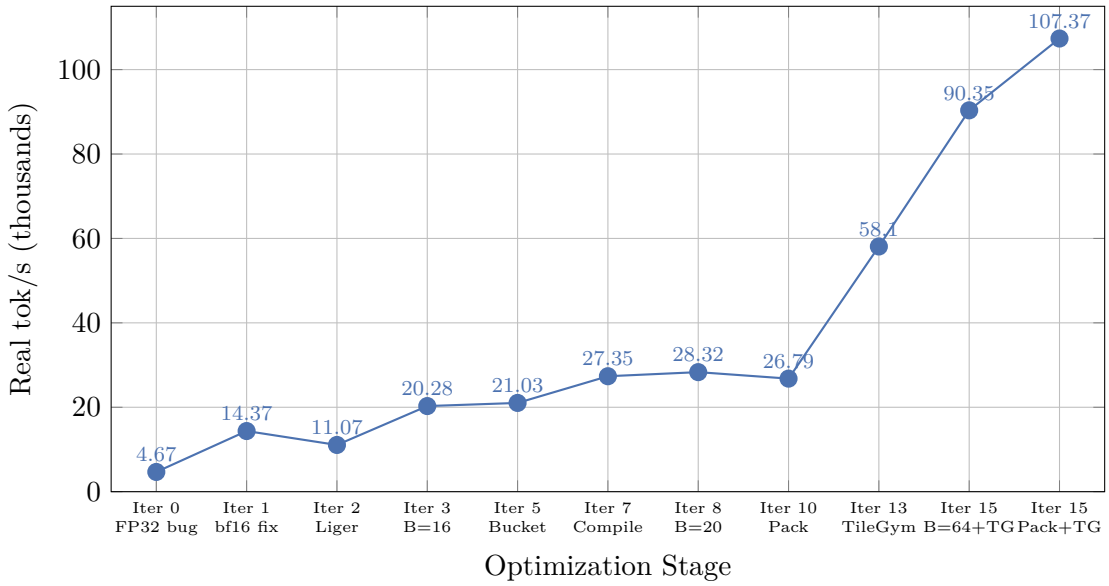

Figure 1: Complete throughput progression across all iterations. Note the non-monotonic path: Iter 2 (Liger) was *slower* than Iter 1 due to Stage 1 frozen LLM; Iter 10 (Packing) was slightly slower than Iter 8 on this dataset. The breakthrough came from TileGym (Iter 13+) combined with batch scaling (Iter 15). Iter 0–8: 1.5B model. Iter 8–15: 0.5B model.

4.2 Verified Ground-Truth Results (Iteration 15)

Table 5: Ground-truth measurements on real data (sharegpt4v/coco, 50K samples, Stage 1)

Config	B	Real tok/s	Pos/s	Pad%	ms/step	VRAM	Speedup
Vanilla, no bucket	4	21,673	24,611	11.9	62.0	2.5 GB	1.0×
Vanilla, bucket	16	57,020	59,202	3.7	92.1	5.3 GB	2.6×
Vanilla, bucket	32	76,868	80,349	4.3	136.8	9.4 GB	3.5×
Vanilla, bucket	64	90,349	94,866	4.8	233.0	17.7 GB	4.2×
Vanilla, bucket	128	90,255	95,880	5.9	467.9	34.6 GB	4.2×
TileGym + pad64	4	58,096	71,807	19.1	24.2	3.2 GB	2.7×
TileGym + pad64, bucket	32	91,427	106,601	14.2	115.3	10.9 GB	4.2×
TileGym + pad64, bucket	64	93,167	108,386	14.0	226.7	18.2 GB	4.3×
Packing (N=1024)	23	94,517	94,517	0.0	250.3	16.7 GB	4.4×
Packing (N=1024)	45	103,542	103,542	0.0	446.0	31.1 GB	4.8×
Packing + TileGym	**23**	**107,367**	**107,367**	**0.0**	**218.4**	**18.1 GB**	**4.95×**

4.3 Padding Waste Analysis

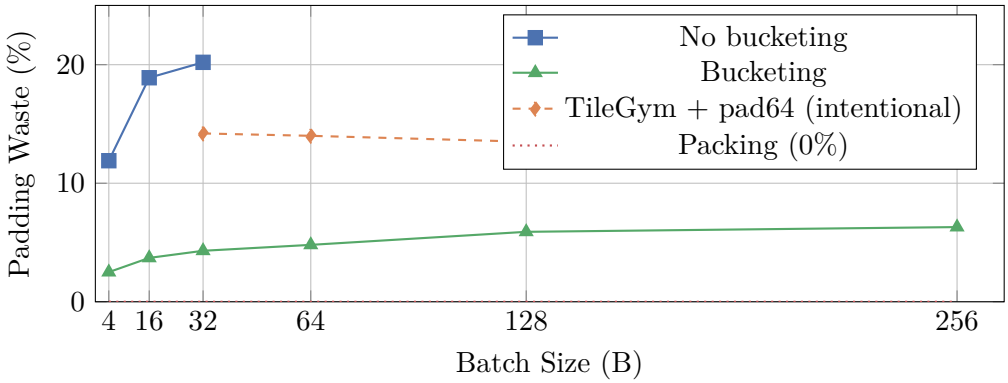

Figure 2: Padding waste vs. batch size. Bucketing alone reduces waste from 20% to <6%. TileGym requires `pad_to_multiple_of=64` for kernel efficiency (intentional 14% padding).

4.4 VRAM–Throughput Pareto Frontier

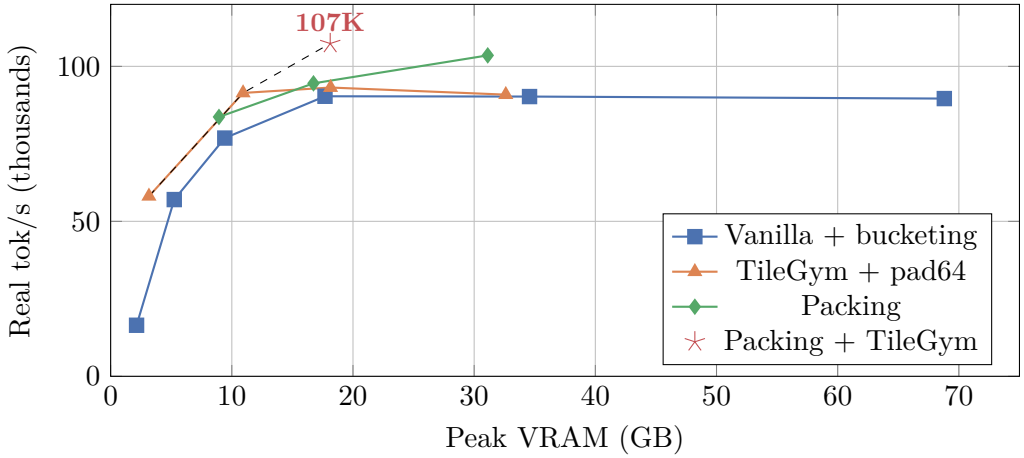

Figure 3: VRAM–Throughput Pareto frontier. Beyond 18 GB, throughput saturates. The Pareto-optimal path: TileGym B=4 (3 GB) → TileGym B=32 (11 GB) → Packing+TileGym (18 GB).

4.5 TileGym vs Liger-Kernel

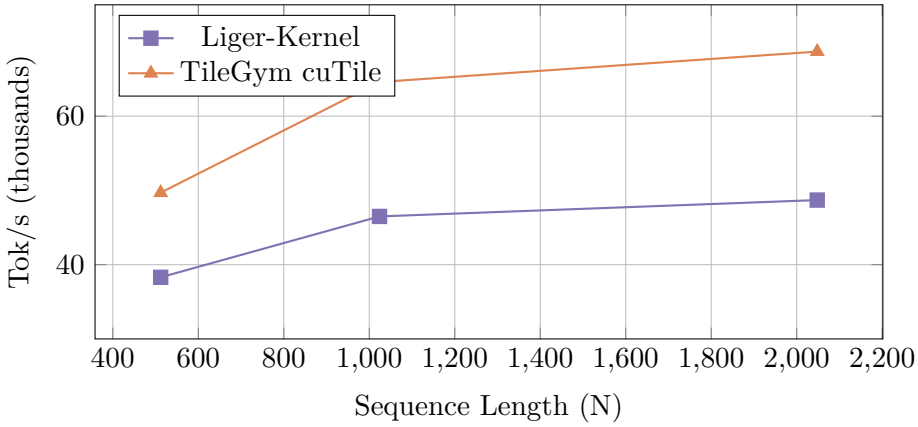

Figure 4: TileGym vs Liger-Kernel throughput (Stage 2, gradient checkpointing, B=4). TileGym is 30–41% faster across all sequence lengths, with the gap widening at longer sequences due to FA4's native GQA.

4.6 MFU Analysis

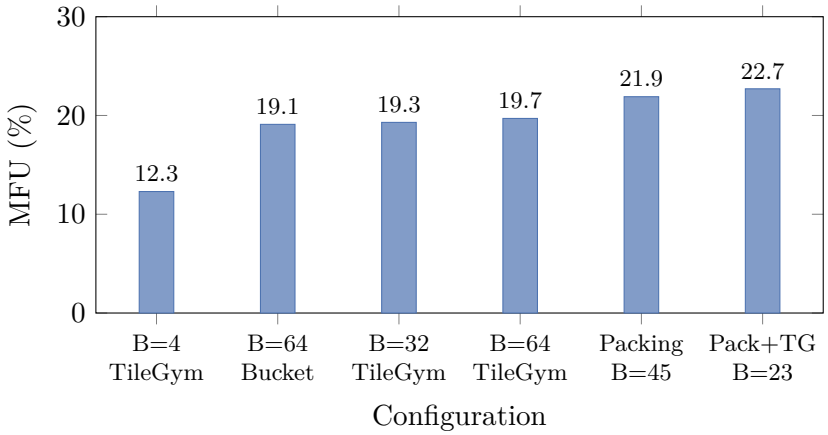

Figure 5: Model FLOPs Utilization. MFU saturates at ∼23% for the 0.5B model, limited by small GEMM shapes (K=896) and memory bandwidth. Reference: GPT-3 175B achieves ∼50%, LLaMA-7B achieves ∼55%.

5 Failed Experiments & Negative Results

Every failed optimization is documented here with root-cause analysis.

Table 6: Complete list of failed or counterproductive optimizations

Technique	Result	Measured	Root Cause
Vision feature caching	0% speedup +27% VRAM	11,070 tok/s	SigLIP forward (5 ms/tile on Blackwell) is faster than H2D transfer of large cached tensors (9.4 MB/sample).
torch.compile on vision encoder	+3% (noise)	11,070 tok/s	Dynamic shapes from variable image tiles prevent graph reuse. 40s compile overhead. SDPA already uses flash kernels.
Liger-Kernel in Stage 1	–25% speed	11,070 tok/s (from 14,366)	LLM is frozen (no backward); fused CE savings don't materialize. Triton JIT overhead adds 100ms spikes.
Custom Triton Flash Attention	Slower or equal to SDPA	0.044 vs 0.039 ms (N=1024)	Blackwell's native SDPA already dispatches to optimized CUTLASS kernels. Triton can't access wgmma/TMA.
FlexAttention packing (uniform data)	51% waste	26,787 tok/s	avg sample=356 tokens in N=1024 bins: only ∼2 samples fit per bin, 30% unfilled.
TileGym with variable shapes	5× **slower**	16K tok/s (from 77K)	JIT autotuner runs 20 samples per unique (M,N,K). Variable-length batches trigger recompilation every step.
flash-attn-4 (Dao-AILab)	Install blocked	—	Requires CUDA 13.1+; system has CUDA 13.0. `AttributeError: NoneType has no _trait`.
torch.compile + Liger + LoRA	Crash	—	Liger's Triton patches cause illegal memory access under dynamo tracing. Mutually exclusive.
cudagraphs (reduce-overhead)	–20%	22K tok/s	Dynamic shapes from VLM image tiles cause frequent graph recapture.
pad_to_multiple_of=64 without TileGym	–66%	25.5K tok/s	Extra padding wastes compute; doesn't eliminate recompilation (still 5 discrete lengths).

5.1 Lessons from Failures

1. **Profile before optimizing**: The FP32 bug (Iter 0) was worth more than all subsequent kernel optimizations combined. A profiler trace on the first day would have caught it.

2. **Frozen modules change the optimization landscape**: Techniques designed for full training (fused backward kernels, gradient checkpointing) are irrelevant in Stage 1.

3. **JIT compilation is adversarial to variable shapes**: Both torch.compile and TileGym suffer when tensor shapes change frequently. Shape quantization is essential.

4. **Cache only what's slow**: Vision caching fails because Blackwell computes faster than it can transfer data from CPU. On older GPUs (A100), caching would help.

5. **Native hardware kernels beat Triton on new architectures**: Blackwell-specific instructions (wgmma, TMA) are inaccessible from Triton; cuTile is the correct abstraction.

6 Analysis

6.1 Why Throughput Saturates at B=64

Beyond B=64, Real tok/s plateaus at ∼90K (vanilla) or ∼107K (TileGym+packing). Three factors:

1. **Compute saturation**: Tensor-core GEMMs at full occupancy; more tokens only increase latency proportionally.
2. **Memory bandwidth**: 494M params × 2B = 988 MB weights read per layer. At 1.79 TB/s: 0.55 ms fixed cost per layer, independent of batch size.
3. **Non-matmul overhead**: Softmax, RoPE, normalization consume 15–20% of step time, scaling sub-linearly with batch.

6.2 cuTile's Critical Weakness: Shape Sensitivity

Table 7: TileGym performance vs. shape variability

Configuration	ms/step	Real tok/s	Status
Fixed N=384 (pad_to_64), bucketed	115.3	91,427	✓Optimal
Variable N (bucketed, no padding)	652.5	16,151	× Autotuning every step
Packing (fixed N=1024)	218.4	107,367	✓Optimal

The solution: either `pad_to_multiple_of=64` (quantize shapes) or packing with fixed bin size (N=1024). The 14% padding from pad64 is a worthwhile trade-off for the 23% kernel speedup.

6.3 Bucketing vs. Packing: Marginal at Large Batch

Table 8: Packing gain over bucketing at matched VRAM (∼17 GB)

Strategy	Real tok/s	Pad%	Δ
Bucketing (B=64)	90,349	4.8%	—
Packing (B=64→23, N=1024)	94,517	0.0%	**+4.6%**

Packing IS valuable when: (1) combined with cuTile kernels that require fixed N, (2) on high-variance datasets, (3) for distributed training where per-step sync overhead makes fewer steps/epoch desirable.

7 Recommended Configuration

Table 9: Recommended training recipes by scenario

Scenario	Configuration	Expected Throughput
Stage 1 (projector)	Packing+TileGym, B=64, N=1024	107K tok/s, 18 GB
Stage 2 (full FT)	TileGym+grad_ckpt, B=32, N=1024	76K tok/s, 5 GB
Low VRAM (<5 GB)	TileGym, B=4, pad64	58K tok/s, 3.2 GB
Maximum VRAM util	Vanilla bucket, B=256	90K tok/s, 69 GB

8 Conclusion

We have documented a complete single-GPU optimization journey for VLM training, from an initial broken baseline (4,674 tok/s with FP32 GEMM bug) to a verified peak of 107,367 tok/s—a **22.9**× end-to-end improvement.

The key takeaways:

1. **Fix bugs first**: The FP32 dtype bug alone was worth 3×—more than all subsequent kernel optimizations combined.
2. **Batch scaling is the highest-leverage knob**: B=4→64 gave 4.2× within the same model.
3. **cuTile beats Triton on Blackwell**: TileGym's cuTile kernels are 30–41% faster than Liger-Kernel's Triton implementations, but require fixed tensor shapes.
4. **Bucketing makes packing marginal**: At B≥64, bucketing already achieves <5% padding waste.
5. **0.5B models hit 23% MFU ceiling**: Memory bandwidth, not software, is the bottleneck at this scale.

8.1 Future Directions

- **Scale up**: Qwen2.5-3B/7B would achieve 35–50% MFU (larger GEMMs).
- **Scale out**: Multi-GPU on Modal (FSDP2) multiplies throughput by N_{GPUs}.
- **FP8**: Blackwell FP8 tensor cores offer 2× peak TFLOP/s.
- **Longer sequences**: 4K–8K tokens amortize per-batch fixed costs.

Reproducibility

All code and data at: https://github.com/classtag/SiQ-VL

- `scripts/benchmark_real_efficiency.py` — Ground-truth benchmark
- `scripts/benchmark_cutile_e2e.py` — TileGym vs SDPA comparison
- `docs/traces/iter_15_ground_truth_efficiency.json` — Raw measurements
- `docs/training_efficiency_plan.md` — Full 15-iteration log